ADR_005 結帳的一致性：單一交易起步

> 狀態：已接受
> 日期：2026-10-14

背景

結帳需要依序與購物車、商品、帳號、結算、折價券、庫存等模組互動（《設計_002_狀態圖與循序圖》3.1）。任何一步失敗，已完成的步驟都必須還原，否則會出現「庫存被預扣但結帳單不存在」之類的不一致。

考慮過的選項

選項	優點	缺點
單一資料庫交易包住所有步驟	由資料庫保證全有或全無；程式簡單；好測試	庫存的鎖持有到交易提交為止，高併發時會排隊；模組在交易上仍綁在一起，日後拆服務或把庫存移到 Redis 時無法成立
補償方式（Saga）	每一步各自提交，鎖持有時間短；模組真正獨立	中間狀態看得到；補償可能失敗，需要重試與冪等；程式中途當機時需要善後機制；複雜度高
分散式交易（兩階段提交）	多個資料庫一起提交	會阻塞、需要協調者、許多元件不支援；業界在服務之間幾乎不用

決定

- 第一版以單一資料庫交易包住結帳的所有步驟，作為正確性與效能的比較基準
- 各模組的介面依補償方式的需求設計：預扣與釋放、鎖定與解除鎖定為成對操作，且皆可安全重試。這些反向操作在取消結帳單時本來就需要
- 庫存預扣放在交易的最後一步（見 ADR_008）

後果

- 好處：第一版能快速做對；壓測時可以量出單一交易在高併發下的瓶頸，作為改用補償方式的依據
- 代價：之後改為補償方式時，訂單模組的協調邏輯需要改寫；其他模組因介面已成對設計，不受影響
- 重新檢討的條件，符合任一項即改為補償方式並新增 ADR：
- 壓測顯示結帳的鎖等待成為瓶頸
- 庫存預扣移出資料庫，例如改用 Redis
- 參與結帳的任一模組拆成獨立服務